AI Security Findings

Overall risk: CRITICAL | Total findings: 20

Executive summary

Risk overview

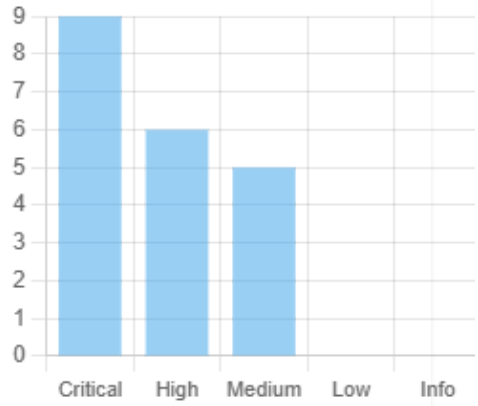
Total findings: 20. Critical: 9, High: 6, Medium: 5, Low: 0.

Technical summary

[
{
"title": "SQL Injection and Training Data Poisoning in store_training_sample",
"severity": "critical",
"location": "store_training_sample, line 18",
"description": "Raw user input concatenated into SQL queries enables SQL injection and allows unvalidated input to poison training data.",
"recommendation": "Use parameterized queries or ORM methods; validate and sanitize inputs to prevent injection and poisoning."
},
{
"title": "Shell Injection and Unvalidated Training Data in fine_tune_on_user_feedback",
"severity": "critical",
"location": "fine_tune_on_user_feedback, line 28",
"description": "User-provided file paths passed unsanitized to subprocess.run with shell=True risk shell injection; unvalidated feedback input may poison fine-tuning.",
"recommendation": "Avoid shell=True; use argument lists in subprocess calls; validate and sanitize feedback files; secure fine-tuning pipelines."
},
{
"title": "Stored Cross-Site Scripting (XSS) via Unsanitized LLM Output in render_llm_response",
"severity": "critical",
"location": "render_llm_response, line 40",
"description": "LLM output is injected directly into HTML without sanitization, allowing stored XSS attacks.",
"recommendation": "Sanitize or escape all LLM output before rendering in HTML contexts."
},
{
"title": "Arbitrary Code Execution via Exec of Unvalidated LLM Output in execute_llm_generated_code",
"severity": "critical",
"location": "execute_llm_generated_code, line 49",
"description": "LLM-generated Python code is executed unsafely with exec(), allowing arbitrary code execution.",
"recommendation": "Avoid exec() on LLM outputs; validate, sandbox, or manually review code before execution."
},
{
"title": "Excessive LLM Agency with Unrestricted Destructive Tools in autonomous_agent_action",
"severity": "critical",
"location": "autonomous_agent_action, line 71",
"description": "LLM agent has unfettered access to destructive functions (email, DB deletion, shell commands, money transfers) with no human oversight.",

"recommendation": "Implement human-in-the-loop reviews, strict access controls, and scope limitations for LLM-triggered actions."
},
{
"title": "Sensitive Information Hardcoded and Leaked via System Prompts",
"severity": "critical",
"location": "global scope and chat_internal, lines 97, 110",
"description": "Secrets such as database credentials and admin passwords are hardcoded and included in system prompts, risking exposure through LLM responses and prompt leaks.",
"recommendation": "Remove all secrets from prompts; store credentials securely; avoid instructing LLM

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Hardcoded OpenAI API key	`global scope, line 11`	The OpenAI API key is hardcoded in the source code, which risks accidental exposure and unauthorized use.	Remove hardcoded API keys from source code and load them securely from environment variables or secret management systems.
CRITICAL	SQL Injection and Training Data Poisoning in store_training_sample	`store_training_sample, line 18`	Raw user input is concatenated directly into SQL query string leading to SQL injection. Additionally, unvalidated input allows data poisoning	Use parameterized SQL queries or ORM methods and validate or sanitize input before storage to prevent injection and poisoning.

Severity	Title	File / Location	Description	Recommendation
			of the training data.	
CRITICAL	Shell Injection and Unvalidated Training Data in fine_tune_on_user_feedback	`fine_tune_on_user_feedback, line 28`	User-controlled file path is passed unsanitized to subprocess.run with shell=True, risking shell injection. User feedback file can contain adversarial content poisoning fine-tuning.	Avoid shell=True and use argument lists in subprocess calls. Validate and sanitize feedback files and use secured pipelines for fine-tuning.
CRITICAL	Stored Cross-Site Scripting (XSS) via Unsanitized LLM Output in render_llm_response	`render_llm_response, line 40`	LLM-generated output is injected directly into HTML response without sanitization, allowing stored XSS attacks.	Sanitize or escape LLM output before rendering in HTML contexts to prevent injection of malicious scripts.
CRITICAL	Arbitrary Code Execution via Exec of Unvalidated LLM Output in execute_llm_generated_code	`execute_llm_generated_code, line 49`	LLM-generated Python code is executed directly using exec() without validation or sandboxing, enabling arbitrary code execution.	Do not execute LLM output directly. Validate, sandbox, or manually review code before execution.
CRITICAL	Command Injection via os.system of LLM-generated Shell Command in run_llm_shell_command	`run_llm_shell_command, line 60`	LLM-generated shell command is executed via os.system without allowlisting or	Implement strict allowlists for shell commands and avoid executing untrusted LLM output directly.

Severity	Title	File / Location	Description	Recommendation
			validation, risking arbitrary command execution.	
CRITICAL	Excessive LLM Agency with Unrestricted Destructive Tools in autonomous_agent_action	`autonomous_agent_action, line 71`	LLM agent is given autonomous access to destructive functions (email, DB record deletion, shell execution, money transfer) with no human confirmation or scope limitation.	Introduce human-in-the-loop review, strict access controls, and scope limitations for LLM-invoked actions.
CRITICAL	High-stakes Decision Delegated Fully to LLM Without Review in auto_approve_llm_decision	`auto_approve_llm_decision, line 84`	LLM makes final binary approval or rejection decisions without any human oversight, allowing unverified, high-impact outcomes.	Require human review or add additional validation layers before automated approval or rejection.
HIGH	Hardcoded Database Connection String with Credentials	`global scope, line 94`	Hardcoded database connection string includes username and password in source code.	Use environment variables or vault solutions to store and access database credentials securely.
MEDIUM	Hardcoded Internal Documentation URL	`global scope, line 95`	Internal documentation URL is hardcoded in source code, potentially exposing	Avoid hardcoding internal URLs; load them securely or restrict source code access.

Severity	Title	File / Location	Description	Recommendation
			internal infrastructure details.	
CRITICAL	System Prompt Leaking Secrets and Instructing LLM to Lie	`global scope, line 97`	System prompt contains confidential information (database credentials, internal URLs, admin password) and instructs the LLM to deny existence of system prompts, exposing secrets if prompt is leaked or repeated.	Remove secrets from system prompts and avoid instructing LLMs to lie. Store secrets securely and separate from prompts.
CRITICAL	Sensitive Data Leakage via System Prompt in chat_internal	`chat_internal, line 110`	System prompt passed to LLM includes sensitive information such as DB credentials and admin passwords, risking exposure through LLM responses.	Remove sensitive credentials from system prompt before sending to LLM.
HIGH	Unvalidated User Embeddings Stored in Shared Namespace Without Access Control in store_user_embedding	`store_user_embedding, line 124`	User content is embedded without validation and stored in a shared vector DB without namespace isolation, enabling poisoning and leakage between users.	Validate embeddings input and segment vector database by namespaces per user or tenant.

Severity	Title	File / Location	Description	Recommendation
MEDIUM	Unbounded Input Length for Embeddings Leading to Potential DoS in get_embedding	`get_embedding, line 133`	Embedding API is called with user input without max length or rate limiting, allowing potential denial of service with large inputs or high request rates.	Enforce input length limits and rate limit requests to embedding endpoints.
HIGH	Lack of Access Control on RAG Query Leading to Data Disclosure in rag_query_no_access_control	`rag_query_no_access_control, line 141`	No per-user access control applied to retrieved documents from vector DB, allowing users to access other users' data.	Implement document-level access control to enforce data isolation between users.
HIGH	Role Misrepresentation in Medical Advice Generation Without Disclaimer in generate_medical_advice	`generate_medical_advice, line 154`	LLM is assigned the role of 'medical doctor' without disclaimers that it is an AI assistant, risking user misinformation and harm.	Include clear disclaimers that model output is not professional medical advice and require review by qualified professionals.
HIGH	Role Misrepresentation in Legal Contract Generation With No Attorney Review in generate_legal_contract	`generate_legal_contract, line 166`	LLM output is presented as legally binding contract text without any attorney oversight, enabling misinformation and liability.	Clearly label generated text as draft only and require legal professional review.

Severity	Title	File / Location	Description	Recommendation
MEDIUM	Lack of max_tokens Parameter Leading to Unbounded Token Usage in process_all_documents	`process_all_documents, line 177`	LLM API calls do not set max_tokens limit, resulting in potential API cost and token exhaustion when processing large document lists.	Enforce max_tokens limits per API call and batch or limit number of documents processed per request.
MEDIUM	Unbounded Loop and Unrestricted Conversation History Growth in unlimited_context_chain	`unlimited_context_chain, line 190`	Conversation history grows indefinitely with no truncation, causing token limit exhaustion or escalating cost.	Implement conversation window size limits or truncation to control token usage.
MEDIUM	Unrestricted Public API Endpoint Without Rate Limiting or max_tokens in api_endpoint_no_rate_limit	`api_endpoint_no_rate_limit, line 203`	API endpoint lacks authentication, rate limiting, and max_tokens setting, enabling API abuse and unbounded token consumption.	Add authentication, rate limits, and enforce max_tokens on LLM API calls.